

ROTAS

Plataforma de Gestao Total de Frotas

Documentacao de Engenharia Completa

Proposta central

Tudo o que acontece com a frota deve ficar registado, controlado, auditavel e disponivel mesmo em ambientes de baixa conectividade.

Versao 1.1 | Mocambique | 2026

Confidencial - Uso Interno

Indice Executivo

1. Visao geral e arquitectura de sistema
2. Stack tecnico recomendado
3. Modelo de dados e multi-tenancy
4. Modulos operacionais
5. API e endpoints
6. App motorista offline-first
7. Alertas e comunicacao
8. Seguranca, auditoria e conformidade
9. Custos, financeiro e modelo de negocio
10. Roadmap, backlog e definition of done
11. Deploy, observabilidade e operacao
12. Riscos e proximos passos

1. Visao Geral e Arquitectura de Sistema

O ROTAS e uma plataforma SaaS para gestao operacional, documental, financeira e auditavel de frotas, desenhada para operadores mocambicanos desde uma unica viatura ate frotas corporativas.

Diferencial competitivo

A plataforma nasce mobile-first e offline-first, com WhatsApp como canal operacional principal, em vez de assumir internet estavel, desktop permanente e grandes equipas administrativas.

Principio	Aplicacao no ROTAS	Justificacao
Modularidade	Modulos activaveis por plano e por maturidade do cliente	Permite vender desde plano Solo ate Enterprise
Offline-first	PWA do motorista guarda dados localmente e sincroniza depois	Conectividade 2G/3G instavel em rotas nacionais
Multi-tenant	Todas as entidades possuem tenant_id e isolamento de permissao	Um unico produto serve varias empresas
Event-driven	Alertas disparados por eventos: checklist, abastecimento, documento, viagem	Gestor recebe excepcao, nao ruido
Auditabilidade	Registos com timestamp, user_id, fotos, GPS e historico	Prova operacional e reducao de fraude
Mobile-first	Fluxos curtos, botoes grandes, camera e GPS	Motoristas usam smartphones Android simples

1.1 Camadas do sistema

- Apresentacao: dashboard gestor em Next.js, PWA motorista em React/Vite e notificacoes WhatsApp.
- API: FastAPI com autenticao JWT, autorizacao por role, validacao Pydantic e rate limiting por tenant.
- Servicos: viaturas, checklists, combustivel, viagens, manutencao, acessorios, motoristas, financeiro e alertas.
- Dados: PostgreSQL/Supabase, Redis, IndexedDB local, Cloudflare R2 para fotos e documentos.

1.2 Recomendacao arquitectural para MVP

Apesar da visao poder evoluir para microservicos, o MVP deve ser um monolito modular FastAPI. Isto reduz complexidade operacional e mantem fronteiras internas claras para futura extraccao.

2. Stack Tecnico Recomendado

Camada	Tecnologia	Motivo
Backend API	FastAPI + Python	Performance, validacao forte, OpenAPI automatico e async nativo
ORM/Migracoes	SQLAlchemy 2.0 async + Alembic	Modelagem robusta, migracoes versionadas
Base de dados	PostgreSQL via Supabase	Managed PostgreSQL, RLS, PostGIS e real-time

Camada	Tecnologia	Motivo
Cache/filas	Redis ou Upstash	Rate limit, jobs leves, cache e eventos
Dashboard	Next.js 14 + TypeScript	Server Components, boa performance e deploy simples
UI	Tailwind + shadcn/ui	Componentes acessiveis e rapidamente personalizaveis
App motorista	PWA React/Vite	Sem app stores, offline-first e baixo atrito
Offline local	IndexedDB + Dexie.js	Persistencia no browser com fila de sincronizacao
Ficheiros	Cloudflare R2	API S3, baixo custo e sem egress fees
Notificacoes	WhatsApp Cloud API	Canal ja usado por motoristas e gestores
Pagamentos	M-Pesa, e-Mola, transferencia	Adequado ao mercado mocambicano

2.1 Estrutura recomendada do backend

- backend/app/main.py - entrada FastAPI
- backend/app/config.py - configuracao por ambiente
- backend/app/database.py - sessoes SQLAlchemy
- backend/app/modules/<modulo>/models.py - modelos ORM
- backend/app/modules/<modulo>/schemas.py - contratos Pydantic
- backend/app/modules/<modulo>/router.py - endpoints REST
- backend/app/modules/<modulo>/service.py - regras de negocio
- backend/app/tasks - jobs, alertas, relatorios e sync
- backend/tests - testes unitarios e integracao

2.2 Decisoese de tecnologia

A prioridade tecnica deve ser confiabilidade, simplicidade operacional e custo baixo. Evitar dependencias complexas antes do MVP validar uso real.

3. Modelo de Dados e Multi-Tenancy

3.1 Estrategia multi-tenant

No MVP, usar tenant_id em todas as tabelas e resolver o tenant a partir do token. RLS no PostgreSQL deve ser activado como camada adicional de defesa quando os fluxos principais estiverem estabilizados.

Entidade	Responsabilidade	Regras criticas
tenants	Empresa/frota cliente	Plano, limites, estado activo, contacto WhatsApp
users	Utilizadores do dashboard	Role, tenant, password hash, estado activo
drivers	Motoristas e equipa operacional	Carta, INATTER, score, viaturas autorizadas

Entidade	Responsabilidade	Regras criticas
vehicles	Ficha central da viatura	Matricula unica por tenant, documentos, estado, QR
checklist_templates	Modelos dinamicos	Itens bloqueantes, foto obrigatoria, categoria
checklists	Execucoes reais	GPS, assinatura, respostas JSONB, estado final
fuel_logs	Abastecimentos	Km crescente, recibo, consumo calculado, anomalias
trips	Viagens	Origem/destino, carga, km, custos, entrega
maintenance_records	Historico tecnico	OS, pecas, custos, bloqueio, downtime
alerts	Notificacoes e eventos	Prioridade, canal, estado, entidade relacionada
files	Fotos e documentos	Storage key, hash, entidade ligada, tenant
audit_logs	Trilha de auditoria	Antes/depois, user, IP, acao, timestamp

3.2 Regras de modelagem

- Todas as tabelas operacionais devem ter tenant_id, created_at e, quando aplicavel, updated_at.
- Registos operacionais devem ser arquivados ou anulados, nao apagados fisicamente.
- Fotos e documentos devem ficar em R2; a base guarda metadados e storage_key.
- Campos flexiveis como respostas de checklist podem usar JSONB, mas relatorios criticos devem ter colunas normalizadas.
- Indices compostos por tenant_id e entidade principal sao obrigatorios para performance.

4. Modulos Operacionais

Modulo 1 - Viaturas & Inventario

- Ficha completa por viatura: marca, modelo, ano, matricula, chassi, cor e categoria.
- Documentos digitais: seguro, INATTER, licenca, DUA e carta de porte.
- Alertas de validade 30, 15 e 7 dias antes.
- QR Code por viatura para abrir checklist correcto.
- Historico de proprietario, operador, estado visual e acessorios vinculados.

Modulo 2 - Checklists Operacionais

- Tipos: pre-partida, chegada, carregamento, entrega, baldeacao, pernoite, semanal e mensal.
- Itens dinamicos por categoria de viatura.
- Foto obrigatoria por item critico, GPS automatico e assinatura digital.
- Itens bloqueantes impedem saida da viatura.
- Funciona offline e sincroniza depois.

Modulo 3 - Combustivel

- Registo de data, hora, posto, litros, valor, km, recibo e motorista.
- Calculo automatico de L/100km e custo por km.
- Alerta de consumo acima da media historica.
- Limite diario por viatura e comparativo entre postos.
- Relatorios por viatura, motorista, rota e periodo.

Modulo 4 - Rotas & Viagens

- Abertura de viagem com origem, destino, motorista, carga, km inicial e documento.
- Paragens: abastecimento, refeicao, avaria, fiscalizacao, pernoite.
- Portagens e travessias: ponte, barcaca, taxas e recibos.
- Fecho com km final, estado da carga, assinatura ou codigo de entrega.
- Relatorio de custo total por viagem.

Modulo 5 - Manutencao

- Planos preventivos por km, dias ou o que vier primeiro.
- Alertas 500km, 200km e no prazo.
- Correctiva com ordem de servico, pecas, oficina, custos e fotos.
- Bloqueio operacional da viatura ate aprovacao.
- Sub-modulo de pneus com posicao, vida util, rotacao e recauchutagem.

Modulo 6 - Acessorios & Equipamentos

- Inventario por viatura: triangulo, extintor, macaco, kit medico, lona, correntes, lacres.
- Estado: OK, danificado, em falta, emprestado, em reparacao.
- Verificacao em checklist de partida e chegada.
- Saida/devolucao com responsavel e alerta de atraso.
- Equipamentos especiais: reefer, grua, cisterna e GPS hardware.

Modulo 7 - Motoristas & Equipa

- Ficha pessoal, contacto de emergencia, carta, INATTE e INSS.
- Tipo: efectivo, prestador ou eventual.
- Viaturas autorizadas a conduzir.
- Score 0-100 baseado em checklists, incidentes, consumo, pontualidade e km sem ocorrencias.
- Historico de multas, acidentes e desempenho.

Modulo 8 - Custos & Financeiro

- Centros de custo por viatura: combustivel, manutencao, pneus, portagens, seguros, multas e despesas.
- Dashboard com custo total mensal, custo/km, viatura mais cara e comparativo mensal.
- Projecao de custos dos proximos 3 meses.
- Exportacao PDF/Excel por viatura, motorista e periodo.
- Base para decisao de substituicao ou abate.

Modulo 9 - Alertas & Comunicacao

- Canal principal: WhatsApp Cloud API.

- Alertas: checklist em atraso, item bloqueante, consumo anormal, documento vencendo, manutencao, incidente e viagem atrasada.
- Centro de notificacoes no dashboard com prioridade.
- Historico de alertas por entidade.
- Configuracao por tenant para evitar excesso de mensagens.

5. API, Offline-First e Seguranca

5.1 Endpoints essenciais

Modulo	Endpoints principais	Observacoes
Auth	POST /login, /refresh, /logout, /register	JWT curto e refresh token rotativo
Vehicles	GET/POST /vehicles, GET/PATCH /vehicles/{id}, /qr-code	Soft delete e limite por plano
Checklists	/templates, POST /checklists, PATCH /{id}, /complete	Suporta execucao offline com idempotency key
Fuel	POST /fuel, /stats, /efficiency, /anomalies	Valida km crescente e anomalias
Trips	POST /trips, /start, /stop, /toll, /complete	Uma viagem activa por viatura/motorista
Maintenance	/plans, /records, /upcoming, /tires	Bloqueio de viatura por OS aberta
Drivers	GET/POST /drivers, /score, /incidents	Dados pessoais protegidos
Financial	/dashboard, /costs, /reports, /projections	Agregacoes por tenant
Alerts	GET /alerts, /read, /dismiss, /settings	Prioridade e canais configuraveis

5.2 Offline-first

- Todas as accoes do motorista devem gravar localmente primeiro.
- A fila de sincronizacao deve usar idempotency_key para evitar duplicacao.
- Fotos devem ser comprimidas localmente antes do upload.
- A app deve mostrar claramente: sincronizado, pendente, erro e a tentar novamente.
- Conflitos simples podem usar last-write-wins; conflitos financeiros devem ir para revisao do gestor.

5.3 Seguranca

- Passwords com bcrypt ou argon2.
- Access token de 15 a 30 minutos; refresh token de 7 dias com rotacao.
- Rate limit em login, upload e endpoints de sync.
- Ficheiros privados em R2 com URLs assinadas.
- Auditoria para alteracoes criticas e exportacao de relatorios.
- Dados pessoais de motoristas com base legal, consentimento e retencao definida.

6. Modelo de Negocio, Roadmap e Operacao

6.1 Pricing

Plano	Target	Viaturas	Preco mensal	Inclui
Solo	Dono de chapa/moto	1-3	400-600 MZN	Viaturas, checklists, combustivel, viagens simples
PME	Transportadora pequena	4-15	250-400 MZN/viatura	Manutencao, motoristas, alertas WhatsApp
Frota	Empresa media	16-50	180-280 MZN/viatura	Financeiro, relatorios, API
Enterprise	Corporativo/Estado	51+	Negociado	SLA, suporte dedicado, customizacoes

6.2 Roadmap recomendado

Fase	Duracao	Entregavel	Criterio de sucesso
MVP	8 semanas	Auth, viaturas, checklists, combustivel e viagens	3 clientes piloto e 50+ checklists reais
v1.0	+6 semanas	Manutencao, acessorios e motoristas	Score inicial e historico tecnico por viatura
v1.5	+4 semanas	Financeiro e relatorios PDF/Excel	Relatorio mensal aceite por cliente piloto
v2.0	+8 semanas	Offline completo e WhatsApp avançado	Zero perda de dados em offline real

6.3 Definition of Done

- Teste unitario e teste de integracao para fluxos principais.
- Validacao de tenant e permissoes em todos os endpoints.
- Estados de loading, vazio e erro no frontend.
- Logs estruturados e auditoria de operacoes criticas.
- Migracao Alembic criada, testada e reversivel quando possivel.
- Documentacao OpenAPI coerente e exemplos de payload.
- Fluxo testado em ecras pequenos e rede lenta.

6.4 Deploy e observabilidade

Componente	Opcao inicial	Notas
Backend	Render, Railway ou Fly.io	Docker, healthcheck e logs estruturados
DB	Supabase PostgreSQL	Backups automaticos, PostGIS e RLS
Redis	Upstash	Rate limit e filas leves
Frontend/PWA	Vercel ou Cloudflare Pages	Deploy rapido e CDN
Storage	Cloudflare R2	Fotos, recibos, documentos e PDFs

Componente	Opcao inicial	Notas
Erros	Sentry	Backend e frontend
Produto	PostHog	Funis, retencao e uso por modulo

7. Riscos, Mitigacoes e Proximos Passos

Risco	Impacto	Mitigacao
Internet instavel	Perda ou atraso de dados	Offline-first desde o MVP e fila de sync robusta
Baixa literacia digital	Motorista abandona app	Fluxos curtos, botoes grandes, linguagem simples e icones
Fraude em combustivel	Perda financeira	Foto recibo, foto odometro, GPS, limites e anomalias
Complexidade excessiva	MVP atrasa	Monolito modular e foco em 4 modulos iniciais
Custo de WhatsApp	Margem reduzida	Alertas configuraveis e envio apenas por excepcao
Dados pessoais	Risco legal	Consentimento, minimizacao, auditoria e retencao definida
Relatorios inconsistentes	Perda de confianca	Agregacoes testadas e dados financeiros normalizados

7.1 Proximos passos praticos

1. Criar repositorio com backend FastAPI e frontend Next.js.
2. Implementar tenants, users, auth e permissoes.
3. Criar migrations iniciais: tenants, users, vehicles, files e audit_logs.
4. Construir modulo de viaturas com upload de fotos/documentos.
5. Criar templates e execucao de checklist pre-partida.
6. Prototipar PWA motorista com armazenamento local.
7. Fechar piloto com 2 a 3 operadores em Maputo.

Recomendacao final

O MVP deve provar controlo operacional real: menos esquecimentos, menos fraude, mais visibilidade e melhor historico por viatura. A tecnologia deve servir esse resultado, nao competir com ele.